

PROJECT BRIEF + STEP-BY-STEP BUILD GUIDE

PaperPilot

Build an AI research-paper assistant with RAG

Summarize papers • Answer questions with citations
Compare findings across multiple research papers

TIME	LEVEL	STACK	COST
3–4 weeks	Intermediate	Python • RAG	Near zero

Complete project details, architecture, working code, evaluation, deployment and presentation tips.

Contents

About this guide . . . 3
1. Project overview . . . 3
2. System architecture . . . 5
3. Tech stack . . . 7
4. Environment setup . . . 8
5. Final project structure . . . 10
Phase 1 – Parse PDFs into clean text . . . 10
Phase 2 – Chunking . . . 12
Phase 3 – Embeddings and the vector store . . . 14
Phase 4 – Retrieval strategy . . . 16
Phase 5 – Grounded generation with citations . . . 17
Phase 6 – Summarization and comparison . . . 19
Phase 8 – Make it robust . . . 21
Phase 9 – Web UI with Streamlit . . . 21
Phase 10 – REST API with FastAPI . . . 24
Phase 11 – Evaluation . . . 25
Phase 12 – Retrieval upgrades (stretch) . . . 28
Phase 13 – Tests, CI and Docker . . . 30
Phase 14 – Present it like a professional . . . 32
Appendix A – Troubleshooting . . . 33
Appendix B – Suggested timeline . . . 34
Appendix C – Learning resources . . . 35
Appendix D – Final checklist . . . 35

About this guide

Read this first, it will save you hours.

Thanks for asking for the project. This guide gives you **everything** you need to build **PaperPilot**, an AI research-paper assistant powered by Retrieval-Augmented Generation (RAG). It covers the idea, the architecture, every design decision, and a complete step-by-step build with working code, followed by evaluation, deployment, and how to present the project so that it genuinely impresses a senior engineer.

Who it's for	Students and early-career engineers who want a portfolio project that goes well beyond a "chat with a PDF" demo.
You should know	Basic Python (functions, classes, pip), the command line, and git. **No prior ML or RAG experience is required**, every concept is explained.
Time needed	About 3–4 weeks part-time (a weekend-sprint MVP is possible, see the timeline in the appendix).
Cost	Near zero. Embeddings run locally for free. The LLM is pay-per-use (a few cents while developing), or use a free/local model, see the LLM note in Phase 5.
How to use it	Build in order. Every phase ends with a **Checkpoint**, do not move on until it passes. Type the code yourself instead of pasting it; you will learn far more.

> **Honest advice before you start**
> A senior engineer is not impressed by a long feature list. They are impressed by **judgment**: you measured quality, you can explain trade-offs, the system fails gracefully, and the code is tested and readable. This guide is built around those signals, so do the evaluation and testing phases even though they are less glamorous than the UI.

1. Project overview

What PaperPilot is, who it's for, and why it's a strong portfolio piece.

The one-line pitch

PaperPilot is an AI research assistant that ingests research-paper PDFs and lets you **summarize** them, **ask questions with verifiable citations**, and **compare findings across multiple papers**, using Retrieval-Augmented Generation so every answer is grounded in the actual text.

The problem

- Reading a paper properly takes 1–3 hours; a literature review needs dozens of them.
- Pasting a PDF into a general chatbot often hallucinates, gives no page references, and breaks on long documents or on more than one paper at a time.
- Comparing methods, datasets, and results across papers is the slowest and most error-prone part of research.

The solution: three core capabilities

Capability	What the user sees	What's technically interesting
Ask with citations	Type a question, get a short answer with numbered references like [2] that open the exact passage, section and page.	Semantic retrieval, metadata filtering, grounded prompting, citation validation, an "I couldn't find that" fallback instead of hallucination.
Structured summary	One click gives a TL;DR plus Problem, Method, Data, Results and Limitations.	Facet-based retrieval (query-focused summarization) so each section is cited and the paper never has to fit in one prompt.
Cross-paper comparison	Pick 2–5 papers and aspects; get a comparison table with citations per cell.	Per-paper retrieval so one long paper cannot crowd out the others; structured table output.

What separates this from a basic chatbot

Basic "chat with PDF"	PaperPilot
Dumps raw text into the prompt	Section-aware parsing and chunking with overlap and rich metadata
One document at a time	Persistent multi-paper library with per-paper filtering
No sources, or made-up ones	Numbered citations that are programmatically validated against retrieved passages
Confidently answers everything	Relevance threshold + explicit "not found" behaviour
"It seems to work"	Evaluation harness: hit@k, MRR and LLM-judged faithfulness on a golden question set
Notebook or one script	Clean package, REST API, web UI, tests, CI, Docker

Skills this project demonstrates

Skill	Where it shows up
Data engineering	PDF parsing, text cleaning, section detection, chunking strategy (Phases 1–2)
ML / NLP fundamentals	Embeddings, cosine similarity, vector search, reranking, hybrid retrieval (Phases 3–4, 12)
LLM application design	Prompt design, grounding, citation handling, summarization, structured outputs (Phases 5–7)
Software engineering	Modular package, config management, REST API, tests, CI, Docker (Phases 8, 10, 11, 13)
Evaluation mindset	Golden set, retrieval metrics, faithfulness checks, ablation experiments (Phase 11)
Product sense	UI for real workflows, graceful failure modes, clear README and demo (Phases 9, 14)

Scope: MVP vs stretch

MVP (build this first)	Stretch (pick 1–2 later)
Upload text-based PDFs • section-aware chunking • local embeddings • ChromaDB • Q&A with citations • structured summary • 2-paper comparison • Streamlit UI • basic tests	Hybrid BM25 + vector search • cross-encoder reranking • import by arXiv ID • streaming answers • table/figure extraction • OCR for scanned PDFs • citation graph • user accounts • observability dashboard

Non-goals: fine-tuning a model, training embeddings, or building a custom vector database. Using good existing components well is the right call, and that is exactly what a senior engineer would do.

2. System architecture

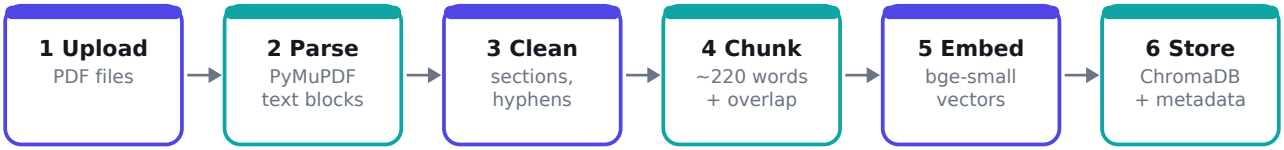
How the pieces fit together, in plain language first, then in detail.

RAG in 60 seconds

A language model knows nothing about *your* PDFs, and 50 papers will not fit in one prompt. **Retrieval-Augmented Generation** solves both problems in two stages:

1. **Index once (offline):** split each paper into small passages (*chunks*), convert each chunk into a vector (an *embedding*) that captures its meaning, and store the vectors in a vector database.
2. **Answer many times (online):** convert the user's question into a vector, find the most similar chunks (*retrieval*), then give only those chunks to the LLM and instruct it to answer from them and cite them (*generation*).

PIPELINE A – INGESTION (RUNS ONCE PER PAPER)



PIPELINE B – QUESTION ANSWERING (RUNS ON EVERY QUESTION)



The data model: what you store for every chunk

Field	Example	Used for
chunk_id	a1b2c3d4e5f6-17	Stable identity; dedupe; evaluation
paper_id	a1b2c3d4e5f6 (hash of file)	Per-paper filtering; avoids re-indexing the same PDF
title	Attention Is All You Need	Citation display; comparison labels
section	Methods	Citation display; future section-aware ranking
page	4	Citation display so users can verify in the PDF
text + embedding	"We apply dropout to..."	What is actually retrieved and sent to the LLM

Key design decisions (and why)

Decision	Choice	Reasoning (be ready to defend this in an interview)
Framework	Plain Python, no LangChain at first	You understand every step and can explain it. Wrapping in a framework later is easy; untangling one is not.
Chunk size	~220 words, 40-word overlap	Big enough to hold a complete idea, small enough to be precise. Overlap protects facts that straddle a boundary. You will tune this with evidence in Phase 11.
Embeddings	BAAI/bge-small-en-v1.5 (local)	Free, fast on CPU, strong on retrieval benchmarks, no data leaves your machine.
Vector DB	ChromaDB (persistent)	Zero-ops, metadata filtering built in, perfect for this scale. Mention FAISS/pgvector/Qdrant as the scale-up path.
Similarity	Cosine on normalized vectors	Standard for text embeddings; scores are interpretable (0–1).
Citations	Numbered passages + regex validation	Cheap, transparent, and testable. You can detect when the model cites a passage that was never provided.
Summaries	Facet-based retrieval, not "stuff the whole PDF"	Cheaper, cited, and scales to long papers; map-reduce is the documented alternative.

3. Tech stack

Every choice has a free or low-cost default and a documented alternative.

Layer	Default choice	Alternatives	Notes
Language	Python 3.11+	—	Type hints throughout.
PDF parsing	PyMuPDF (import fitz)	pdfplumber, GROBID, pymupdf4llm	Fast, gives text blocks with positions and types.
Embeddings	sentence-transformers + bge-small-en-v1.5	OpenAI / Voyage / Cohere embeddings, bge-base, e5	Local = free + private.
Vector store	ChromaDB	FAISS, Qdrant, pgvector, Weaviate	Metadata filter used for per-paper search.
LLM	Anthropic Claude (API)	OpenAI, Gemini, Groq, local via Ollama	Isolated behind one function so it is swappable.
Backend API	FastAPI + Uvicorn	Flask	Auto-generated docs at /docs.
UI	Streamlit	React/Next.js + the FastAPI backend	Streamlit is the fastest route to a polished demo.
Retrieval upgrades	rank-bm25, CrossEncoder reranker	Cohere Rerank, ColBERT	Phase 12 (stretch).

Layer	Default choice	Alternatives	Notes
Testing / CI	pytest + GitHub Actions	—	Run on every push.
Packaging	Docker + docker-compose	—	One-command run for reviewers.

4. Environment setup

Phase 0 – about 20 minutes.

Prerequisites

- Python 3.11 or newer (`python3 --version`), git, and a code editor (VS Code recommended).
- About 2 GB of free disk (the embedding model and its dependencies).
- An LLM API key. This guide uses Anthropic's API, create a key in the Anthropic Console and keep it secret. Free alternatives are covered in Phase 5.
- A handful of test PDFs. Use open-access arXiv papers, e.g. *Attention Is All You Need*, the original RAG paper, *Lost in the Middle*, LoRA, and BERT: a nice set of 5 related papers for the comparison feature.

Create the project

terminal

```
mkdir paperpilot-project && cd paperpilot-project
git init
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
mkdir -p paperpilot tests eval data/papers
touch paperpilot/__init__.py
```

requirements.txt

```
pymupdf
chromadb
sentence-transformers
anthropic
python-dotenv
streamlit
fastapi
uvicorn
python-multipart
rank-bm25
pytest
```

terminal

```
pip install -r requirements.txt
```

.env

```
ANTHROPIC_API_KEY=your-key-here
LLM_MODEL=claude-sonnet-5-5
EMBED_MODEL=BAAI/bge-small-en-v1.5
```

.gitignore

```
.venv/
.env
data/
__pycache__/
.pytest_cache/
```

Never commit your API key

Add **.env** to **.gitignore** before your first commit. If a key ever reaches GitHub, revoke it immediately, deleting the commit is not enough. Commit a **.env.example** with placeholder values instead.

Configuration lives in one place so every other module stays clean:

paperpilot/config.py

```
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()


@dataclass(frozen=True)
class Settings:
    llm_model: str = os.getenv("LLM_MODEL", "claude-sonnet-5-5")
    embed_model: str = os.getenv("EMBED_MODEL", "BAAI/bge-small-en-v1.5")
    chroma_dir: str = os.getenv("CHROMA_DIR", "data/chroma")
    upload_dir: str = os.getenv("UPLOAD_DIR", "data/papers")
    top_k: int = int(os.getenv("TOP_K", "6"))
    chunk_words: int = int(os.getenv("CHUNK_WORDS", "220"))
    chunk_overlap: int = int(os.getenv("CHUNK_OVERLAP", "40"))


settings = Settings()
```

Checkpoint 0

Running `python -c "from paperpilot.config import settings; print(settings)"` prints your settings without errors.

5. Final project structure

Where everything will live by the end.

layout

```
paperpilot-project/
|-- paperpilot/
|   |-- __init__.py
|   |-- config.py        # settings from env vars
|   |-- ingest.py        # PDF -> paragraphs -> chunks            (Phase 1-2)
|   |-- store.py         # embeddings + ChromaDB wrapper          (Phase 3)
|   |-- prompts.py       # system prompts, context, citations     (Phase 5)
|   |-- llm.py           # the ONLY file that talks to the LLM    (Phase 5)
|   `-- service.py       # PaperPilot class: ask/summarize/compare (Phase 4-7)
|-- streamlit_app.py     # web UI                                  (Phase 9)
|-- api.py               # FastAPI REST API                        (Phase 10)
|-- eval/
|   |-- golden.jsonl     # question + expected evidence            (Phase 11)
|   `-- run_eval.py
|-- tests/
|   |-- test_ingest.py
|   `-- test_prompts.py
|-- data/                # gitignored: uploaded PDFs + vector index
|-- Dockerfile  docker-compose.yml  requirements.txt
|-- .github/workflows/tests.yml
`-- README.md
```

Notice the separation of concerns: **parsing**, **storage**, **prompting** and the **LLM client** are independent modules. Swapping the LLM provider touches one file; changing the vector DB touches one file. That is the kind of structure reviewers look for.

Phase 1 – Parse PDFs into clean text

Goal: turn a research-paper PDF into clean paragraphs tagged with section and page.

Why this phase matters more than people expect

Garbage in, garbage out. Research papers are hard: two-column layouts, headers and footers, hyphenated line breaks, equations, and a reference list that pollutes search results. If parsing is sloppy, no amount of clever retrieval will save the answers. We use **PyMuPDF**, which returns text *blocks* with coordinates, and we sort them into reading order.

What the code does

1. Opens the PDF and computes a **content hash** as the paper ID (re-uploading the same file never duplicates data).
2. Guesses the title from PDF metadata, falling back to the first long text block on page 1.
3. Reads text blocks page by page with `sort=True` (fixes most two-column ordering issues).
4. Cleans each block: removes hyphenated line breaks, joins lines, collapses whitespace.

5. Detects headings such as Abstract, Introduction, Methods, Results and tracks the **current section**.
6. **Stops at References** so bibliography text never gets indexed.

paperpilot/ingest.py (part 1 of 2)

```
import hashlib
import re
from dataclasses import dataclass
from pathlib import Path

import fitz  # PyMuPDF

from .config import settings

HEADING = re.compile(
    r"^\s*(\d+(\.\d+)*\.?\s+)?(abstract|introduction|related work|background|"
    r"methods?|methodology|approach|experiments?|results|discussion|limitations|"
    r"conclusions?|references)\s*$", re.I)


@dataclass
class Chunk:
    chunk_id: str
    paper_id: str
    title: str
    section: str
    page: int
    text: str


def file_id(path: str) -> str:
    return hashlib.sha1(Path(path).read_bytes()).hexdigest()[:12]


def clean(text: str) -> str:
    text = text.replace("-\n", "")          # de-hyphenate line breaks
    text = re.sub(r"\s*\n\s*", " ", text)   # newlines -> spaces
    return re.sub(r"\s{2,}", " ", text).strip()


def guess_title(doc, path: str) -> str:
    meta = ((doc.metadata or {}).get("title") or "").strip()
    if len(meta) > 8:
        return meta
    for blk in doc[0].get_text("blocks", sort=True):
        t = clean(blk[4])
        if blk[6] == 0 and 15 < len(t) < 200:
            return t
    return Path(path).stem
```

paperpilot/ingest.py (part 1 continued)

```
def parse_pdf(path: str):
    """Return (paper_id, title, [(paragraph_text, section, page_number), ...])."""
    doc = fitz.open(path)
    pid, title = file_id(path), guess_title(doc, path)
    paragraphs, section, done = [], "Front Matter", False
    for pno, page in enumerate(doc, start=1):
        for blk in page.get_text("blocks", sort=True):
            txt, kind = blk[4], blk[6]
            if kind != 0:            # 0 = text block, 1 = image block
                continue
            txt = clean(txt)
            m = HEADING.match(txt)
            if m:
                section = m.group(3).title()
                if section == "References":
                    done = True
                    break
                continue
            if len(txt) >= 25:       # drop page numbers, stray labels
                paragraphs.append((txt, section, pno))
        if done:
            break
    return pid, title, paragraphs
```

Known limitations (say these out loud in interviews)

Heading detection by regex misses unusual headings; titles are heuristic; equations and tables come out as messy text; a hyphen that is part of a real word at a line break ("state-of-\nthe-art") is joined wrongly; scanned PDFs have no text layer at all. Document these in your README, then list "OCR + layout-aware parsing (GROBID / pymupdf4llm)" as future work. Knowing your limits reads as seniority.

Checkpoint 1

In a Python shell, run `pid, title, paras = parse_pdf('data/papers/yourpaper.pdf')`. Print the title, the first 5 paragraphs and the set of sections. You should see sensible sections and **no reference-list text**. Fix the regex until your 5 test papers look right.

Phase 2 – Chunking

Goal: split paragraphs into retrieval-sized passages without losing meaning.

The concept

An embedding represents one passage as one vector. Too large a chunk blurs several ideas together and retrieval gets vague; too small and a chunk loses the context needed to be useful. Research papers behave well at roughly **150–300 words**. We also add a **40-word overlap** between neighbouring chunks so a sentence that straddles a boundary still appears whole in at least one chunk.

Rules our chunker follows

- **Never cross a section boundary**, a chunk from Methods should not contain half of Results.
- Pack whole paragraphs together until the word budget is reached; hard-split only paragraphs longer than the budget.
- Carry an overlap tail into the next chunk *within* a section (not across sections).
- Record the **page** where each chunk starts and its **section**, they power the citations.
- Drop tiny leftovers (< 30 words) that carry no meaning on their own.

paperpilot/ingest.py (part 2 of 2)

```
def chunk(paragraphs, pid, title, target=None, overlap=None):
    target = target or settings.chunk_words
    overlap = overlap or settings.chunk_overlap
    chunks, buf, new, page, section = [], [], 0, None, None

    def flush(keep_tail=True):
        nonlocal buf, new, page
        if new > 0 and len(buf) >= 30:        # 'new' avoids emitting a pure-overlap chunk
            n = len(chunks)
            chunks.append(Chunk(f"{pid}-{n}", pid, title, section, page, " ".join(buf)))
        buf = buf[-overlap:] if keep_tail else []
        new, page = 0, None

    for txt, sec, pno in paragraphs:
        if sec != section:                    # new section: close the current chunk
            flush(keep_tail=False)
            section = sec
        words = txt.split()
        for i in range(0, len(words), target):
            piece = words[i:i + target]
            if buf and len(buf) + len(piece) > target:
                flush()
            if page is None:
                page = pno
            buf.extend(piece)
            new += len(piece)
    flush(keep_tail=False)
    return chunks


def ingest_pdf(path: str) -> list[Chunk]:
    pid, title, paragraphs = parse_pdf(path)
    return chunk(paragraphs, pid, title)
```

Checkpoint 2

Chunk a real paper and print: number of chunks (typically 40–120 for a conference paper), average words per chunk (close to 200), and three random chunks. Read them. Each should make sense on its own. If chunks look cut off mid-thought, revisit paragraph handling.

Phase 3 – Embeddings and the vector store

Goal: make chunks searchable by meaning.

The concept

An **embedding model** maps text to a list of numbers (here, 384 of them) so that passages with similar meaning end up close together. "How does the model avoid overfitting?" lands near a chunk about dropout and regularization even though they share almost no words, something keyword search cannot do. **Cosine similarity** measures closeness: 1.0 means the same direction, 0 means unrelated.

Details that matter

- **Normalize** embeddings so cosine similarity is a simple dot product and Chroma's cosine index behaves.
- BGE models are trained with a **query instruction prefix**: prepend it to questions, but not to documents. Forgetting this silently hurts retrieval quality.
- Use `upsert` with deterministic chunk IDs so re-indexing is idempotent.
- Store metadata (paper_id, title, section, page) next to each vector, it enables filtering and citations.
- Chroma returns *distance*; convert to similarity with `1 - distance`.

paperpilot/store.py (part 1 of 2)

```
import chromadb
from sentence_transformers import SentenceTransformer

from .config import settings

QUERY_PREFIX = "Represent this sentence for searching relevant passages: "
_model = None


def embedder() -> SentenceTransformer:
    global _model
    if _model is None:                          # lazy-load: the first call downloads ~130 MB
        _model = SentenceTransformer(settings.embed_model)
    return _model


def embed_docs(texts):
    return embedder().encode(texts, normalize_embeddings=True, batch_size=32).tolist()


def embed_query(q):
    return embedder().encode(QUERY_PREFIX + q, normalize_embeddings=True).tolist()


def _where(paper_ids):
    return {"paper_id": {"$in": paper_ids}} if paper_ids else None
```

paperpilot/store.py (part 2 of 2)

```
class PaperStore:
    def __init__(self):
        self.client = chromadb.PersistentClient(path=settings.chroma_dir)
        self.col = self.client.get_or_create_collection(
            "papers", metadata={"hnsw:space": "cosine"})

    def has_paper(self, paper_id: str) -> bool:
        return len(self.col.get(where={"paper_id": paper_id}, limit=1)["ids"]) > 0

    def add_chunks(self, chunks):
        self.col.upsert(
            ids=[c.chunk_id for c in chunks],
            documents=[c.text for c in chunks],
            embeddings=embed_docs([c.text for c in chunks]),
            metadatas=[{"paper_id": c.paper_id, "title": c.title,
                        "section": c.section, "page": c.page} for c in chunks],
        )

    def search(self, query: str, k: int = 6, paper_ids=None):
        res = self.col.query(query_embeddings=[embed_query(query)], n_results=k,
                             where=_where(paper_ids))
        return [{"id": res["ids"][0][i], "text": res["documents"][0][i],
                 "meta": res["metadatas"][0][i], "score": 1 - res["distances"][0][i]}
                for i in range(len(res["ids"][0]))]

    def get_chunks(self, paper_ids=None):
        r = self.col.get(where=_where(paper_ids), include=["documents", "metadatas"])
        return [{"id": i, "text": t, "meta": m, "score": 0.0}
                for i, t, m in zip(r["ids"], r["documents"], r["metadatas"])]

    def list_papers(self):
        seen = {}
        for m in self.col.get(include=["metadatas"])["metadatas"]:
            p = seen.setdefault(m["paper_id"], {"paper_id": m["paper_id"],
                                                "title": m["title"], "chunks": 0})
            p["chunks"] += 1
        return list(seen.values())

    def delete_paper(self, paper_id: str):
        self.col.delete(where={"paper_id": paper_id})
```

Checkpoint 3

Index two papers, then run `PaperStore().search('what dataset was used for evaluation?', k=3)`. Read the three returned passages: they should be genuinely about evaluation data, with scores roughly 0.45–0.8. Then add `paper_ids=[one_id]` and confirm results come only from that paper. Restart Python and search again, results should persist.

Phase 4 – Retrieval strategy

Goal: reliably fetch the right evidence, and know when there isn't any.

Retrieval is where most RAG projects quietly succeed or fail. The basic version is already in `PaperStore.search`; here we add the decisions around it.

Four decisions to make deliberately

Decision	Our default	Trade-off
How many chunks (k)	6 for Q&A, 3–4 per paper for comparison	Too few misses evidence; too many dilutes the prompt and raises cost. Models also attend poorly to the middle of long contexts.
Relevance threshold	Drop hits with score < 0.30	Prevents answering from irrelevant text. Tune it on your golden set, it depends on the embedding model.
Per-paper filtering	Metadata $in filter on paper_id	Lets users scope questions; essential for fair comparisons.
Follow-up questions	Rewrite into a standalone query using chat history	"What about its limitations?" retrieves garbage unless you resolve "its". Costs one small LLM call.

The query rewriter is a tiny but high-impact function. Without it, multi-turn chat feels broken:

paperpilot/service.py (helper, shown here for context)

```
REWRITE_SYSTEM = ("Rewrite the user's last question as a standalone search query, resolving "
                  "pronouns using the conversation. Output only the query, nothing else.")


def standalone(history, question: str) -> str:
    if not history:
        return question
    convo = "\n".join(f"{role}: {text}" for role, text in history[-6:])
    return complete(REWRITE_SYSTEM, f"{convo}\nuser: {question}", max_tokens=80).strip()
```

Checkpoint 4

Ask 10 questions you know the answers to. For each, check whether the right chunk appears in the top 6 and note where. Write down what fails, that list becomes the seed of your golden evaluation set in Phase 11.

Phase 5 – Grounded generation with citations

Goal: answers that cite their sources and refuse to make things up.

The prompting strategy

- Give the model **numbered passages** with title, section and page, and require every claim to carry a citation like [2].
- State an **explicit fallback**: if the context lacks the answer, say so, verbatim, instead of guessing.
- Forbid inventing numbers, datasets or references; require numbers to be copied exactly.
- Keep the question **after** the context; keep instructions in the system prompt.
- **Validate the output in code**: extract every [n], confirm it maps to a real passage, and flag any that do not.

paperpilot/prompts.py

```
import re

NOT_FOUND = "I couldn't find that in the selected papers."

SYSTEM = f"""You are PaperPilot, a careful research assistant.
Rules:
1. Answer ONLY from the numbered context passages provided.
2. Cite every factual claim with its passage number in square brackets, e.g. [2] or [1][3].
3. If the context does not contain the answer, reply exactly: "{NOT_FOUND}"
4. Never invent numbers, datasets, authors or citations. Copy numbers exactly as written.
5. Be concise: short paragraphs or bullets."""

COMPARE_SYSTEM = SYSTEM + """
Task: compare the papers on the requested aspects. Output a markdown table with one row per
aspect and one column per paper, citing passages like [3] inside cells. Write 'Not stated'
when a paper's passages do not cover an aspect. After the table add 2-3 sentences on the
most important differences."""

FACETS = {
    "Problem and motivation": "What problem does the paper address and why does it matter?",
    "Method": "What is the proposed method or approach? Describe the key components.",
    "Data and experiments": "Which datasets, baselines and evaluation metrics were used?",
    "Key results": "What are the main quantitative results?",
    "Limitations": "What limitations or future work do the authors mention?",
}

CITE = re.compile(r"\[(\d+)\]")


def format_context(hits) -> str:
    blocks = []
    for i, h in enumerate(hits, 1):
        m = h["meta"]
        blocks.append(f"[{i}] ({m['title']} | {m['section']} | p.{m['page']})\n{h['text']}")
    return "\n\n".join(blocks)


def parse_citations(text: str, hits):
    used = sorted({int(n) for n in CITE.findall(text)})
```

```
    valid = [n for n in used if 1 <= n <= len(hits)]
    invalid = [n for n in used if n not in valid]
    return valid, invalid
```

The LLM client: one small file, easy to swap

Everything that touches the model lives in this file. To use OpenAI, Gemini, Groq or a local model via Ollama, rewrite only these two functions, nothing else changes.

paperpilot/llm.py

```
import anthropic

from .config import settings

_client = None


def client():
    global _client
    if _client is None:
        _client = anthropic.Anthropic()       # reads ANTHROPIC_API_KEY from the environment
    return _client


def complete(system: str, user: str, max_tokens: int = 1200) -> str:
    msg = client().messages.create(
        model=settings.llm_model, max_tokens=max_tokens, system=system,
        messages=[{"role": "user", "content": user}])
    return "".join(b.text for b in msg.content if b.type == "text")


def stream(system: str, user: str, max_tokens: int = 1200):
    with client().messages.stream(
            model=settings.llm_model, max_tokens=max_tokens, system=system,
            messages=[{"role": "user", "content": user}]) as s:
        yield from s.text_stream
```

Free or local LLM option

No API budget? Run a local model with Ollama and call its OpenAI-compatible endpoint, or use a free-tier provider. Smaller models follow citation rules less reliably, which makes your validation and evaluation code even more valuable, and gives you a great story: "I compared model sizes on faithfulness."

Checkpoint 5

Wire `search → format_context → complete` by hand in a shell. Ask something the paper answers (you should get cited text) and something it does not, like "What is the capital of France?" (you should get the exact fallback sentence). Both behaviours must work before you continue.

Phase 6 – Summarization and comparison

Goal: the two features that make PaperPilot more than Q&A.

Summaries: query-focused instead of "stuff everything"

Sending an entire paper to the model works for short ones but is expensive, slow, and unciteable. Instead we ask five fixed questions (the *facets* in prompts.py), retrieve the best 4 chunks per facet, and answer each in 2–4 sentences. Each section keeps its own sources. A final call merges them into a 3-sentence TL;DR. The documented alternative is **map-reduce**: summarize every chunk, then summarize the summaries, better coverage, higher cost, weaker citations. Mention both in your README.

Comparison: retrieve per paper, then synthesize

If you search all papers at once, the paper with the most matching text dominates the results and the comparison becomes lopsided. So for every aspect we retrieve **k chunks from each paper separately**, merge and de-duplicate them, number them, and ask the LLM for a markdown table with citations inside cells.

The service class that ties everything together

The UI and the API both call this one class. That is why we kept the modules separate.

paperpilot/service.py (part 1 of 2)

```
from .config import settings
from .ingest import ingest_pdf
from .llm import complete
from .prompts import (COMPARE_SYSTEM, FACETS, NOT_FOUND, SYSTEM, format_context,
                      parse_citations)
from .store import PaperStore

MIN_SCORE = 0.30
REWRITE_SYSTEM = ("Rewrite the user's last question as a standalone search query, resolving "
                  "pronouns using the conversation. Output only the query, nothing else.")


def standalone(history, question: str) -> str:
    if not history:
        return question
    convo = "\n".join(f"{role}: {text}" for role, text in history[-6:])
    return complete(REWRITE_SYSTEM, f"{convo}\nuser: {question}", max_tokens=80).strip()


class PaperPilot:
    def __init__(self):
        self.store = PaperStore()

    # ---- indexing ------------------------------------------------------------
    def add_pdf(self, path: str) -> dict:
        chunks = ingest_pdf(path)
```

```
        if not chunks:
            raise ValueError("No extractable text found. Is this a scanned PDF?")
        pid = chunks[0].paper_id
        if not self.store.has_paper(pid):
            self.store.add_chunks(chunks)
        return {"paper_id": pid, "title": chunks[0].title, "chunks": len(chunks)}

    # ---- question answering ------------------------------------------------
    def ask(self, question: str, paper_ids=None, history=None) -> dict:
        query = standalone(history, question)
        hits = self.store.search(query, k=settings.top_k, paper_ids=paper_ids)
        hits = [h for h in hits if h["score"] >= MIN_SCORE]
        if not hits:
            return {"answer": NOT_FOUND, "sources": [], "invalid_citations": [], "query": query}
        user = f"CONTEXT:\n{format_context(hits)}\n\nQUESTION: {question}"
        answer = complete(SYSTEM, user)
        _, invalid = parse_citations(answer, hits)
        return {"answer": answer, "sources": hits, "invalid_citations": invalid, "query": query}
```

paperpilot/service.py (part 2 of 2, indented inside the class)

```
    # ---- summarization -----------------------------------------------------
    def summarize(self, paper_id: str) -> dict:
        sections = []
        for name, question in FACETS.items():
            hits = self.store.search(question, k=4, paper_ids=[paper_id])
            user = (f"CONTEXT:\n{format_context(hits)}\n\nQUESTION: {question}\n"
                    "Answer in 2-4 sentences.")
            sections.append((name, complete(SYSTEM, user, max_tokens=500), hits))
        digest = "\n\n".join(f"{n}: {t}" for n, t, _ in sections)
        tldr = complete(
            "Write a 3-sentence TL;DR of a research paper from these notes. "
            "Do not include citation numbers.", digest, max_tokens=300)
        return {"tldr": tldr, "sections": sections}

    # ---- comparison --------------------------------------------------------
    def compare(self, paper_ids, aspects) -> dict:
        found = {}
        for aspect in aspects:
            for pid in paper_ids:                       # per-paper retrieval = fair comparison
                for h in self.store.search(aspect, k=3, paper_ids=[pid]):
                    found[h["id"]] = h                  # dict de-duplicates overlapping hits
        hits = list(found.values())
        titles = sorted({h["meta"]["title"] for h in hits})
        user = (f"CONTEXT:\n{format_context(hits)}\n\nPAPERS: {'; '.join(titles)}\n"
                f"ASPECTS: {', '.join(aspects)}\nProduce the comparison.")
        answer = complete(COMPARE_SYSTEM, user, max_tokens=1800)
        _, invalid = parse_citations(answer, hits)
        return {"answer": answer, "sources": hits, "invalid_citations": invalid}
```

Checkpoint 6–7

From a Python shell: `pp = PaperPilot(); pp.add_pdf(...)` for two papers, then call `pp.summarize(pid)` and `pp.compare([pid1, pid2], ['method','datasets','results'])`. Verify at least five claims in the output against the PDFs by hand. If a claim is wrong, look at the cited passage, was it retrieved badly, or did the model misread it? Those are different bugs with different fixes.

Phase 8 – Make it robust

Goal: the boring parts that separate a demo from software.

Risk	What to do
Scanned or empty PDFs	Already raise ValueError in add_pdf. Show a friendly message in the UI. Stretch: OCR with ocrmypdf or pytesseract.
Huge uploads	Limit file size (e.g. 25 MB) and page count in the API and UI; reject others with a clear error.
LLM errors / rate limits	Wrap calls with retry + exponential backoff (e.g. the tenacity library); surface a readable error, not a stack trace.
Duplicate uploads	Content hash as paper_id + has_paper check, already done.
Hallucinated citations	parse_citations returns invalid numbers; show a warning badge in the UI when any exist.
Prompt injection inside PDFs	Treat PDF text as **data, not instructions**. The system prompt says to answer only from context; keep it in the system role and never let retrieved text override rules.
Secrets	Keys only via environment variables; .env gitignored; never log full prompts containing user documents in production.
Logging	Log timings for parse, embed, retrieve, generate, plus token counts. You will need them for the cost and latency table in Phase 14.

paperpilot/llm.py (retry wrapper)

```
from tenacity import retry, stop_after_attempt, wait_exponential


@retry(stop=stop_after_attempt(4), wait=wait_exponential(multiplier=1, min=1, max=20))
def complete(system: str, user: str, max_tokens: int = 1200) -> str:
    ...  # same body as before; add `tenacity` to requirements.txt
```

Phase 9 – Web UI with Streamlit

Goal: a clean interface with sidebar library, chat, summary and comparison tabs.

Streamlit reruns your script top to bottom on every interaction, so keep expensive objects behind `@st.cache_resource` and conversation state in `st.session_state`. Put the file in the project root so the `paperpilot` package is importable.

streamlit_app.py (part 1 of 2)

```
import os

import streamlit as st

from paperpilot.config import settings
from paperpilot.service import PaperPilot

st.set_page_config(page_title="PaperPilot", layout="wide")


@st.cache_resource
def get_app() -> PaperPilot:
    return PaperPilot()


pp = get_app()
st.session_state.setdefault("history", [])
st.session_state.setdefault("indexed", set())


def show_sources(hits, invalid=()):
    if invalid:
        st.warning(f"Model cited passage(s) {list(invalid)} that were not "
                   "retrieved. Verify manually.")
    for i, h in enumerate(hits, 1):
        m = h["meta"]
        label = (f"[{i}] {m['title'][:48]} | {m['section']} | "
                 f"p.{m['page']} | score {h['score']:.2f}")
        with st.expander(label):
            st.write(h["text"])


# ------------------------------ sidebar: library ------------------------------
with st.sidebar:
    st.header("Library")
    for f in st.file_uploader("Add PDFs", type="pdf", accept_multiple_files=True) or []:
        if f.name in st.session_state.indexed:
            continue
        with st.spinner(f"Indexing {f.name}..."):
            os.makedirs(settings.upload_dir, exist_ok=True)
            path = os.path.join(settings.upload_dir, f.name)
            with open(path, "wb") as out:
                out.write(f.getbuffer())
            try:
                pp.add_pdf(path)
                st.session_state.indexed.add(f.name)
            except ValueError as e:
                st.error(f"{f.name}: {e}")
    papers = pp.store.list_papers()
    titles = {p["paper_id"]: p["title"] for p in papers}
    st.caption("Search scope")
    selected = [p["paper_id"] for p in papers
                if st.checkbox(p["title"][:55], value=True, key=p["paper_id"])]
```

streamlit_app.py (part 2 of 2)

```
st.title("PaperPilot")
tab_ask, tab_sum, tab_cmp = st.tabs(["Ask", "Summarize", "Compare"])

# ---------------------------------- Ask -----------------------------------
with tab_ask:
    for role, text in st.session_state.history:
        with st.chat_message(role):
            st.markdown(text)
    if q := st.chat_input("Ask about the selected papers..."):
        prior = list(st.session_state.history)
        st.session_state.history.append(("user", q))
        with st.chat_message("user"):
            st.markdown(q)
        with st.chat_message("assistant"):
            with st.spinner("Searching and reading..."):
                res = pp.ask(q, selected or None, history=prior)
            st.markdown(res["answer"])
            show_sources(res["sources"], res["invalid_citations"])
        st.session_state.history.append(("assistant", res["answer"]))

# ---------------------------------- Summarize ----------------------------------
with tab_sum:
    if not papers:
        st.info("Upload a paper in the sidebar to begin.")
    else:
        pid = st.selectbox("Paper", list(titles), format_func=lambda x: titles[x])
        if st.button("Generate summary"):
            with st.spinner("Reading the paper..."):
                out = pp.summarize(pid)
            st.subheader("TL;DR")
            st.write(out["tldr"])
            for name, text, hits in out["sections"]:
                st.markdown(f"**{name}**")
                st.write(text)
                show_sources(hits)

# ---------------------------------- Compare ----------------------------------
with tab_cmp:
    aspects = st.text_input("Aspects (comma-separated)",
                            "problem, method, datasets, results, limitations")
    if len(selected) < 2:
        st.info("Select at least two papers in the sidebar.")
    elif st.button("Compare papers"):
        with st.spinner("Comparing..."):
            res = pp.compare(selected, [a.strip() for a in aspects.split(",") if a.strip()])
        st.markdown(res["answer"])
        show_sources(res["sources"], res["invalid_citations"])
```

terminal

```
streamlit run streamlit_app.py
```

Checkpoint 9

Upload two PDFs, ask a question, ask a follow-up using "it" or "that method", generate a summary, run a comparison. Then try to break it: upload a scanned PDF, ask something off-topic, deselect every paper. Fix whatever crashes. Record a 60–90 second screen capture, it becomes your demo.

Phase 10 – REST API with FastAPI

Goal: expose the same engine as a service so any client can use it.

A UI-only project looks like a notebook with buttons. A documented API shows you can build systems other people integrate with. Because all logic is in `PaperPilot`, the API is thin.

api.py

```
import os
import shutil

from fastapi import FastAPI, File, HTTPException, UploadFile
from pydantic import BaseModel, Field

from paperpilot.config import settings
from paperpilot.service import PaperPilot

app = FastAPI(title="PaperPilot API", version="1.0")
pp = PaperPilot()
MAX_MB = 25


class AskIn(BaseModel):
    question: str = Field(min_length=3)
    paper_ids: list[str] | None = None
    history: list[tuple[str, str]] = []


class CompareIn(BaseModel):
    paper_ids: list[str] = Field(min_length=2, max_length=5)
    aspects: list[str] = ["problem", "method", "datasets", "results", "limitations"]


@app.post("/papers")
async def upload(file: UploadFile = File(...)):
    if not (file.filename or "").lower().endswith(".pdf"):
        raise HTTPException(400, "Only PDF files are supported.")
    os.makedirs(settings.upload_dir, exist_ok=True)
    path = os.path.join(settings.upload_dir, os.path.basename(file.filename))
    with open(path, "wb") as out:
        shutil.copyfileobj(file.file, out)
    if os.path.getsize(path) > MAX_MB * 1024 * 1024:
        os.remove(path)
        raise HTTPException(413, f"File larger than {MAX_MB} MB.")
    try:
        return pp.add_pdf(path)
    except ValueError as e:
        raise HTTPException(422, str(e))


@app.get("/papers")
def list_papers():
    return pp.store.list_papers()


@app.delete("/papers/{paper_id}")
def delete_paper(paper_id: str):
    pp.store.delete_paper(paper_id)
    return {"deleted": paper_id}
```

```
@app.post("/ask")
def ask(body: AskIn):
    return pp.ask(body.question, body.paper_ids, body.history)


@app.get("/papers/{paper_id}/summary")
def summary(paper_id: str):
    return pp.summarize(paper_id)


@app.post("/compare")
def compare(body: CompareIn):
    return pp.compare(body.paper_ids, body.aspects)
```

terminal

```
uvicorn api:app --reload
# open http://127.0.0.1:8000/docs for interactive Swagger documentation
curl -F "file=@data/papers/attention.pdf" http://127.0.0.1:8000/papers
curl -X POST http://127.0.0.1:8000/ask -H "Content-Type: application/json" \
     -d '{"question": "What optimizer and learning-rate schedule were used?"}'
```

Checkpoint 10

Open **/docs**, upload a PDF, list papers, ask a question, and compare two papers, all from the browser. A screenshot of that Swagger page belongs in your README.

Phase 11 – Evaluation

Goal: replace "it seems to work" with numbers. This is the phase that impresses seniors most.

Two questions need separate answers, because they fail for different reasons:

- **Retrieval quality:** did the right passage come back? Metrics: **hit@k** (is any relevant chunk in the top k?) and **MRR** (mean reciprocal rank, rewards ranking the right chunk higher).
- **Answer quality:** is the answer **faithful** to the retrieved text (no unsupported claims) and does the system correctly say "not found" for unanswerable questions?

Step 1 – Build a golden set (30–50 questions)

Pick 5 papers. For each, write 6–10 questions you can answer from the paper, and for each question copy one or two **distinctive phrases** from the passage that answers it (the "evidence"). Add 5–10 **unanswerable** questions ("What GPU did they train on?" when the paper never says). Mix easy factual questions with harder ones that need reasoning over a method description.

eval/golden.jsonl (one JSON object per line)

```
{"question": "Which optimizer?", "paper_id": "a1b2c3d4e5f6", "evidence": ["Adam"], "answerable": true}
{"question": "Which dataset?", "paper_id": "a1b2c3d4e5f6", "evidence": ["WMT"], "answerable": true}
{"question": "How many A100 GPUs?", "paper_id": "a1b2c3d4e5f6", "evidence": [], "answerable": false}
```

Step 2 – Measure retrieval and faithfulness

eval/run_eval.py (part 1 of 2)

```
import json

from paperpilot.config import settings
from paperpilot.llm import complete
from paperpilot.prompts import NOT_FOUND, SYSTEM, format_context
from paperpilot.store import PaperStore

JUDGE = ("You are a strict fact-checker. Decide whether every factual claim in the ANSWER is "
         "supported by the CONTEXT. Reply with JSON only: "
         '{"supported": true, "unsupported_claims": []}')


def load(path="eval/golden.jsonl"):
    return [json.loads(line) for line in open(path) if line.strip()]


def retrieval_eval(k=None):
    k, store = k or settings.top_k, PaperStore()
    rows = [e for e in load() if e["answerable"]]
    hits_k, rr = 0, 0.0
    for ex in rows:
        hits = store.search(ex["question"], k=k, paper_ids=[ex["paper_id"]])
        rank = next((i for i, h in enumerate(hits, 1)
                     if any(p.lower() in h["text"].lower() for p in ex["evidence"])), None)
        if rank:
            hits_k += 1
            rr += 1 / rank
    print(f"retrieval  n={len(rows)}  hit@{k}={hits_k/len(rows):.2f}  MRR={rr/len(rows):.2f}")
```

eval/run_eval.py (part 2 of 2)

```
def answer_eval(limit=30):
    store, faithful, refused_ok, n_ans, n_un = PaperStore(), 0, 0, 0, 0
    for ex in load()[:limit]:
        hits = store.search(ex["question"], k=settings.top_k, paper_ids=[ex["paper_id"]])
        ctx = format_context(hits)
        out = complete(SYSTEM, f"CONTEXT:\n{ctx}\n\nQUESTION: {ex['question']}")
        if ex["answerable"]:
            n_ans += 1
            raw = complete(JUDGE, f"CONTEXT:\n{ctx}\n\nANSWER:\n{out}", max_tokens=400)
            try:
                verdict = json.loads(raw.strip().strip("`").removeprefix("json"))
                faithful += bool(verdict["supported"])
            except Exception:
                pass                              # unparseable judge output counts as a miss
        else:
            n_un += 1
            refused_ok += NOT_FOUND in out
    print(f"faithful={faithful}/{n_ans}   correct 'not found'={refused_ok}/{n_un}")


if __name__ == "__main__":
    retrieval_eval()
    answer_eval()
```

Step 3 – Run experiments and keep a results table

Change **one thing at a time**, rerun, record. This table is the centerpiece of your README. Fill in your own numbers, never copy anyone else's.

Experiment	Hit@6	MRR	Faithful	Latency	Takeaway
Baseline (220 words, bge-small)	—	—	—	—	
Chunk size 120 / 400 words	—	—	—	—	
Overlap 0 vs 40 vs 80	—	—	—	—	
Embedding: bge-base vs bge-small	—	—	—	—	
k = 3 / 6 / 10	—	—	—	—	
+ Hybrid BM25 + vector (Phase 12)	—	—	—	—	
+ Cross-encoder rerank (Phase 12)	—	—	—	—	
Without query rewriting (multi-turn set)	—	—	—	—	

Be honest about the eval's limits

A 40-question set you wrote yourself is small and may be biased toward easy phrasing. Say so in the README and note how you would improve it (more papers, questions written by someone else, adversarial questions). The **RAGAS** library offers ready-made RAG metrics once you are comfortable with the basics. An LLM judge is itself imperfect: spot-check 10 of its verdicts by hand.

Checkpoint 11

You have a golden set, a baseline row in the results table, and at least two experiments with a written conclusion such as "chunk size 400 lowered MRR because chunks mixed unrelated topics."

Phase 12 – Retrieval upgrades (stretch)

Goal: improve the numbers you just measured, with evidence.

Upgrade 1: Hybrid search (BM25 + vectors)

Vector search is great at meaning but can miss exact tokens such as model names, dataset names, equation symbols and acronyms. **BM25** is classic keyword ranking that nails those. Combine both rankings with **Reciprocal Rank Fusion**: each result earns 1/(60 + rank) from every list it appears in.

paperpilot/hybrid.py

```
from rank_bm25 import BM25Okapi


def rrf(rank_lists, k=60):
    scores = {}
    for ranks in rank_lists:
        for pos, doc_id in enumerate(ranks, 1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + pos)
    return sorted(scores, key=scores.get, reverse=True)


def hybrid_search(store, query, paper_ids=None, k=6):
    docs = store.get_chunks(paper_ids)
    bm25 = BM25Okapi([d["text"].lower().split() for d in docs])
    sc = bm25.get_scores(query.lower().split())
    bm_rank = [docs[i]["id"] for i in sorted(range(len(docs)), key=lambda i: -sc[i])[:20]]
    vec_rank = [h["id"] for h in store.search(query, k=20, paper_ids=paper_ids)]
    by_id = {d["id"]: d for d in docs}
    return [by_id[i] for i in rrf([bm_rank, vec_rank])[:k]]
```

Note: fused hits carry `score = 0`, so skip the MIN_SCORE filter when using hybrid (or apply it to the vector list before fusing). Rebuilding BM25 on every query is fine for a few thousand chunks; cache the index per paper set if it gets slow.

Upgrade 2: Cross-encoder reranking

Embedding search compares a query and a passage *separately*. A **cross-encoder** reads them *together* and scores relevance far more accurately, but is slower, so use it on a shortlist: retrieve 20, rerank, keep the best 5.

paperpilot/rerank.py

```
from sentence_transformers import CrossEncoder

_reranker = None


def rerank(query, hits, top_n=5):
    global _reranker
    if _reranker is None:
        _reranker = CrossEncoder("BAAI/bge-reranker-base")
    scores = _reranker.predict([(query, h["text"]) for h in hits])
    ranked = sorted(zip(scores, hits), key=lambda x: x[0], reverse=True)
    return [h for _, h in ranked[:top_n]]
```

Wire either upgrade into `PaperPilot.ask` behind a config flag, rerun your evaluation, and put the before/after numbers in your README.

More upgrade ideas, roughly by impact

Idea	How	Why it's impressive
Streaming answers	`st.write_stream(stream(SYSTEM, user))` after retrieving hits	Feels instant; shows you understand latency.
Import by arXiv ID	Download *arxiv.org/pdf/<id>* with urllib, then call add_pdf; respect arXiv's rate guidance	Removes friction; great demo moment.
Better parsing	pymupdf4llm or GROBID for layout-aware structure, tables and references	Tackles the real weakness of RAG: input quality.
OCR for scans	ocrmypdf as a preprocessing step when no text is found	Handles messy real-world input.
Citation graph	Query the Semantic Scholar API for references and citations; link related papers in the UI	Turns a Q&A tool into a research tool.
Caching	Cache embeddings by text hash and LLM answers by (question, chunk ids)	Cost and latency awareness.
Observability	Log traces to Langfuse or LangSmith; track latency, tokens, cost per request	Production mindset.
Auth + per-user libraries	Add a user_id metadata field and filter every query by it; JWT auth in FastAPI	Multi-tenant design thinking.

Phase 13 – Tests, CI and Docker

Goal: anyone can clone, run, and trust your project in minutes.

Unit tests for the logic that doesn't need an API key

Test pure functions: cleaning, chunking, and citation parsing. LLM calls and embeddings are slow and non-deterministic, so test those through the evaluation harness instead (and mention that distinction).

tests/test_ingest.py

```
from paperpilot.ingest import chunk, clean


def test_clean_joins_hyphenated_line_breaks():
    assert clean("learn-\ning is fun") == "learning is fun"


def test_chunks_never_cross_sections_and_respect_budget():
    paras = [("word " * 300, "Introduction", 1), ("word " * 100, "Methods", 2)]
    chunks = chunk(paras, "p1", "Title", target=220, overlap=40)
    assert {c.section for c in chunks} == {"Introduction", "Methods"}
    assert all(len(c.text.split()) <= 300 for c in chunks)


def test_chunk_records_starting_page():
    paras = [("word " * 100, "Methods", 7)]
    assert chunk(paras, "p1", "T")[0].page == 7
```

tests/test_prompts.py

```
from paperpilot.prompts import format_context, parse_citations

HITS = [{"text": "alpha", "meta": {"title": "T1", "section": "Methods", "page": 3}},
        {"text": "beta", "meta": {"title": "T2", "section": "Results", "page": 5}}]


def test_valid_and_invalid_citations_are_separated():
    valid, invalid = parse_citations("Fact A [1]. Fact B [2][5].", HITS)
    assert valid == [1, 2] and invalid == [5]


def test_context_is_numbered_with_page_info():
    ctx = format_context(HITS)
    assert ctx.startswith("[1] (T1 | Methods | p.3)") and "[2] (T2 | Results | p.5)" in ctx
```

Continuous integration

.github/workflows/tests.yml

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install pytest pymupdf python-dotenv
      - run: pytest -q
```

Docker

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN python -c "from sentence_transformers import SentenceTransformer as S; \
    S('BAAI/bge-small-en-v1.5')"
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "streamlit_app.py", "--server.address=0.0.0.0"]
```

docker-compose.yml

```
services:
  ui:
    build: .
    ports: ["8501:8501"]
    env_file: .env
    volumes: ["./data:/app/data"]
```

Run it with `docker compose up --build`. The volume keeps your index between restarts.

Deploying a public demo

- **Streamlit Community Cloud** or **Hugging Face Spaces**: easiest free options; store the API key in their secrets manager.
- **Render / Fly.io / Railway:** deploy the Docker image; attach a persistent volume for `data/`.
- **Protect your wallet:** a public demo with your API key can be abused. Add upload-size limits, a per-session question cap, and a spend limit in your LLM provider's console. Or pre-load 3–5 sample papers and disable uploads on the public build.

Licensing note

Only index papers you are allowed to use. Open-access arXiv papers are fine for personal projects, but do not commit PDFs to your repo or redistribute them; the **data/** folder is gitignored for this reason. Link to the papers instead.

Phase 14 – Present it like a professional

A great build with a weak presentation gets ignored.

README structure that works

1. **Title, one-line pitch, and a demo GIF** at the very top (60–90 seconds: upload, ask, cite, compare).
2. **Why** it exists: the problem in two sentences.
3. **Architecture diagram** (the two pipelines from Section 2) and a short explanation.
4. **Features** and a **quick start** that works in under 5 commands (docker compose up).
5. **Evaluation results table** with your real numbers and the conclusions you drew.
6. **Design decisions and trade-offs:** chunking, embeddings, why no framework, hybrid/rerank findings.
7. **Limitations and future work:** scanned PDFs, tables/equations, small eval set, prompt-injection risk.
8. Tech stack, project layout, how to run tests, license.

Repo hygiene checklist

- Small, meaningful commits with clear messages ("Add section-aware chunking"), not one giant commit.
- Type hints and short docstrings on public functions; consistent formatting (ruff or black).
- Green CI badge, a **.env.example**, no secrets or PDFs in the repo, a pinned **requirements.txt** (`pip freeze`).
- A pinned issue list showing your roadmap ("Add OCR", "Evaluate bge-base"), it shows you think beyond version one.

Resume bullets (replace [X] with your real measured numbers)

- Built **PaperPilot**, a RAG research assistant (Python, FastAPI, ChromaDB, sentence-transformers, Streamlit) that answers questions with validated citations, generates structured summaries and compares findings across papers.
- Designed section-aware chunking and per-paper filtered retrieval; improved hit@6 from [X] to [Y] using hybrid search and cross-encoder reranking, measured on a [N]-question golden set.
- Implemented citation validation and a not-found fallback; achieved [X]% LLM-judged faithfulness and [Y]% correct refusals on unanswerable questions.
- Shipped a documented REST API, pytest suite, GitHub Actions CI and a Docker image.

Questions a senior engineer will ask (and what a strong answer contains)

Question	Strong answer includes
Why RAG instead of fine-tuning?	Knowledge changes constantly; citations and verifiability; no training data or GPU cost; fine-tuning teaches style, not facts.
How did you choose the chunk size?	You ran experiments (show the table), and can explain the precision-vs-context trade-off and why overlap helps.
How do you prevent hallucinations?	You can't fully eliminate them; you reduce and detect them: grounding prompt, relevance threshold, not-found fallback, citation validation, faithfulness eval.
Retrieval returns bad results. Debug it.	Separate retrieval from generation failures; inspect retrieved chunks first; then parsing quality, chunking, query rewriting, hybrid search, reranking.
How would this scale to millions of chunks?	Managed vector DB (pgvector, Qdrant), HNSW ANN indexes, async ingestion queue, batched GPU embeddings, caching, per-tenant filtering.
What are the security risks?	Prompt injection through PDF text, PII in uploads, API-key leakage, abusive uploads, cost-based denial of service; describe your mitigations.
Why cosine similarity and normalized vectors?	Direction matters more than magnitude for text embeddings; normalized cosine equals dot product and is what the models are trained for.
What would you do next?	Layout-aware parsing for tables, a larger independent eval set, answer-level citation checking, user feedback loop.

Appendix A – Troubleshooting

Symptoms you are likely to meet and where to look first.

Symptom	Likely cause	Fix
add_pdf raises "No extractable text"	Scanned PDF (images only)	Run ocrmypdf on the file first, or use a text-based version of the paper.
Garbled or interleaved sentences	Two-column layout read out of order	Keep sort=True; try pymupdf4llm or GROBID; inspect blocks with page.get_text('blocks').
References appear in answers	Heading not detected so indexing never stopped	Print sections per paper; extend the HEADING regex (e.g. "Bibliography", "Literature Cited").
Retrieval returns irrelevant chunks	Missing BGE query prefix, chunks too large, or poor parse	Confirm embed_query adds the prefix; inspect chunks; try 150-word chunks; add rewriting or rerank.
Follow-up questions fail	No query rewriting	Pass history to ask(); verify res['query'] is a standalone question.
Model cites [7] but only 6 passages exist	Hallucinated citation	Already flagged by invalid_citations; tighten the prompt; try a stronger model.

Symptom	Likely cause	Fix
Always answers "I couldn't find that"	MIN_SCORE too high, or wrong paper filter	Print scores; lower the threshold; check selected paper_ids.
First run is very slow	Embedding model downloading and loading	Normal once (~130 MB); preload in Docker build; cache with st.cache_resource.
Chroma error on where filter	Empty list passed to $in	Pass None when no papers are selected (already handled with `selected or None`).
Streamlit: ModuleNotFoundError paperpilot	App not run from project root	Run from the root folder; keep streamlit_app.py beside the package.
Rate-limit or timeout errors	Too many LLM calls (summaries make 6)	Add retry with backoff; cache results; run facets concurrently with a thread pool.

Appendix B – Suggested timeline

Two ways to pace the build.

Four-week plan (part-time, ~8–10 hours a week)

Week	Focus	You finish with
1	Phases 0–3: setup, parsing, chunking, embeddings and vector store	Index 5 papers and see good search results in a shell
2	Phases 4–7: retrieval choices, grounded answers, summaries, comparison	Working core engine with cited answers; start writing golden questions
3	Phases 8–10: robustness, Streamlit UI, FastAPI	A usable app and API; demo recording
4	Phases 11–14: evaluation, one upgrade (hybrid or rerank), tests, CI, Docker, README	Polished repo with a results table and live demo link

Weekend-sprint MVP

- **Day 1:** Phases 0–5. End the day answering cited questions about one paper from a shell.
- **Day 2:** Phases 6–7 and 9. Add summary and comparison, build the Streamlit UI, write a short README, push to GitHub. Return later for evaluation and tests.

Appendix C – Learning resources

Search these by name; all are free to read.

Topic	Resource
RAG foundations	"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020)
Context-use pitfalls	"Lost in the Middle: How Language Models Use Long Contexts" (Liu et al., 2023)
RAG evaluation	"RAGAS: Automated Evaluation of Retrieval Augmented Generation" (Es et al., 2023) and the RAGAS docs
Embeddings	Sentence-Transformers documentation; the BGE / FlagEmbedding model cards on Hugging Face; the MTEB leaderboard
Vector search	ChromaDB docs; the HNSW paper (Malkov and Yashunin); FAISS wiki
Hybrid retrieval	Reciprocal Rank Fusion (Cormack et al., 2009); rank-bm25 README
PDF parsing	PyMuPDF documentation (text extraction recipes); pymupdf4llm; GROBID
Building with LLMs	Anthropic documentation (Messages API, prompt engineering guides); your chosen provider's equivalent
Apps and APIs	Streamlit docs (session state, caching); FastAPI tutorial
Good test corpus	arXiv: Attention Is All You Need, BERT, the RAG paper, LoRA, Lost in the Middle

Appendix D – Final checklist

Tick these off before you share the project.

- ■ Parsing: titles, sections and pages look right on 5 different papers; references are excluded
- ■ Chunking: sections respected, overlap works, chunk stats documented
- ■ Retrieval: per-paper filter, relevance threshold, query rewriting for follow-ups
- ■ Answers: citations present, invalid citations flagged, "not found" fallback verified
- ■ Summaries and comparisons: five claims manually verified against the PDFs
- ■ UI and API: both work end to end; errors are friendly; Swagger screenshot captured
- ■ Evaluation: golden set, baseline numbers, at least 2 experiments with conclusions
- ■ Quality: tests pass locally and in CI; Docker image runs from a clean clone
- ■ Security: no secrets or PDFs in git; upload limits; spend limit set on the API key
- ■ Presentation: README with demo GIF, architecture diagram, results table, limitations; resume bullets use real numbers

Final word

Build the MVP first, then **measure, improve and explain**. A smaller project with an honest evaluation, clear trade-offs and clean code will beat a sprawling one that nobody can verify. Make it your own: pick a research domain you care about (medical, climate, law, finance), tailor the prompts and test papers to it, and tell that story in your README. Good luck, and have fun building it.